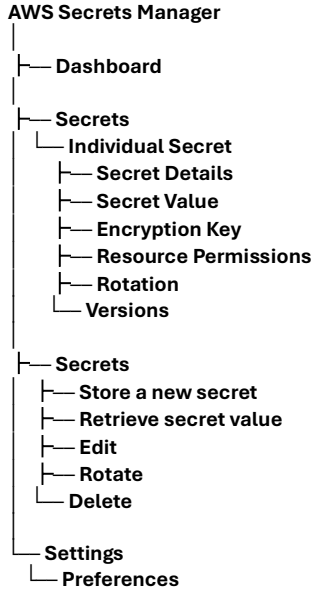
Secrets Manager — Console Dashboard Wise Important Interview Questions & Answers

AWS Secrets Manager



1. What is AWS Secrets Manager?
Answer:
AWS Secrets Manager is a service used to securely store sensitive information such as database passwords, API keys, and application credentials.
Applications can retrieve these secrets when they need them instead of keeping passwords directly in the code.
It also supports secret rotation, encryption, and access control using IAM.
For example:
Suppose our application connects to an RDS database and needs a username and password.
Instead of putting the password inside the application code, we store it in Secrets Manager.
The application retrieves the secret securely when it needs to connect to RDS.
Example:
Application
↓
Secrets Manager
↓
Username + Password
↓
RDS Database
Simple ga gurthupettuko:
Secrets Manager = Securely store and manage passwords/secrets

---

2. What is a Secret in AWS Secrets Manager?
Answer:
A Secret is a stored piece of sensitive information managed by Secrets Manager.
It can contain database credentials, API keys, tokens, or other confidential values.
Each secret has its own name and can be accessed by authorized applications or users.
For example:
Suppose our application needs an RDS MySQL username and password.
We create a secret such as prod/database and store the credentials inside it.
The application can then retrieve this secret instead of storing the password in code.
Example:
Secrets Manager
↓
prod/database
↓
Username + Password
Simple ga gurthupettuko:
Secret = Securely stored sensitive information

---

3. What is Secret Value?
Answer:
Secret Value is the actual sensitive information stored inside a secret.
It can contain values such as username, password, API key, token, or connection information.
Secrets Manager encrypts the secret value so that it is protected while stored.

For example:
Suppose we create a secret for an RDS database.
The secret value can contain the database username and password.
Our application retrieves these values when it needs database access.
Example:
Secret
↓
Secret Value
├— username
└— password
Simple ga gurthupettuko:
Secret Value = Actual password/key stored inside the secret

---

4. How do you create a Secret from the AWS Console?
Answer:
In the Secrets Manager console, we select **Store a new secret**.
Then we choose the type of secret and enter the required credentials or key-value information.
After that, we provide a secret name, configure encryption and optional rotation, and create the secret.
For example:
Suppose we want to store an RDS database password.
We open Secrets Manager → Store a new secret → select database credentials → enter the credentials → give the secret a name.
Finally, we create the secret and allow the application to access it using IAM.
Example:
Secrets Manager
↓
Store a new secret
↓
Credentials
↓
Secret Name
↓
Create Secret
Simple ga gurthupettuko:
Store a new secret = Create and securely store credentials

---

5. What is Secret Rotation?
Answer:
Secret Rotation means automatically changing a secret value periodically.
It is commonly used for database passwords so that old credentials do not remain valid for a long time.
Secrets Manager can perform rotation using supported AWS integrations and Lambda-based rotation workflows.
For example:
Suppose an application uses an RDS database password stored in Secrets Manager.
Instead of keeping the same password permanently, we configure rotation.
Secrets Manager changes the credentials periodically and updates the required database/application configuration.
Example:
Old Password
↓
Rotation
↓
New Password
↓
Application + Database
Simple ga gurthupettuko:
Rotation = Automatically change the secret

---

6. What is the Encryption Key in Secrets Manager?
Answer:
Secrets Manager encrypts secret values using AWS Key Management Service (KMS).
By default, Secrets Manager can use an AWS managed KMS key for encryption.
We can also use a customer managed KMS key when we need more control over encryption permissions.
For example:
Suppose we store an RDS password in Secrets Manager.
The password is encrypted using a KMS key while stored.
Only authorized identities with the required permissions can retrieve and decrypt the secret.
Example:
Secret Value
↓
KMS Encryption
↓
Encrypted Secret
↓

Secrets Manager
Simple ga gurthupettuko:
KMS = Encrypt and protect Secrets Manager data

---

7. How does IAM control access to Secrets Manager?
Answer:
IAM policies control which users, roles, or applications can access secrets.
We can allow specific actions such as retrieving, creating, updating, or deleting secrets.
This follows the principle of least privilege, where applications receive only the permissions they need.
For example:
Suppose an EC2 application needs to read an RDS password from Secrets Manager.
We attach an IAM role to EC2 and allow that role to retrieve only the required secret.
The application can access the password without giving it full Secrets Manager permissions.
Example:
EC2
↓
IAM Role
↓
secretsmanager:GetSecretValue
↓
Secret
Simple ga gurthupettuko:
IAM = Decide who can access the secret

---

8. What is Resource-based Permission in Secrets Manager?
Answer:
A resource-based policy can be attached to a secret to control who can access that particular secret.
It is useful when we need to grant access to specific AWS accounts, roles, or identities.
This can be useful for cross-account access scenarios.
For example:
Suppose our production secret is owned by Account A, but an application in Account B needs to retrieve it.
We can use appropriate IAM and resource-based permissions to allow the required cross-account access.
Only the specified identity should be allowed to retrieve the secret.
Example:
Account A
↓
Secret
↓
Resource Policy
↓
Account B Application
Simple ga gurthupettuko:
Resource Policy = Control access directly on the secret

---

9. What are Secret Versions?
Answer:
Secrets Manager maintains versions when the value of a secret changes.
Different versions can have different secret values during updates or rotation.
Version stages help identify which version is currently active and which version is being used during rotation.
For example:
Suppose we rotate an RDS password.
The old password belongs to an older secret version and the new password becomes the current version.
Secrets Manager uses version stages to manage these versions during the rotation process.
Example:
Secret
↓
Version 1 → Old Password
↓
Version 2 → New Password
Simple ga gurthupettuko:
Versions = Different values of the same secret over time

---

10. What is the difference between Secrets Manager and storing passwords in application code?
Answer:
Storing passwords directly in application code is insecure because developers, source repositories, or logs may expose them.
Secrets Manager provides encrypted storage and controlled access through IAM.
It also supports rotation, making secret management easier and safer.
For example:
Instead of writing an RDS password directly inside an application configuration file, we store it in Secrets Manager.
The EC2 application uses its IAM role to retrieve the password when required.
This keeps sensitive credentials outside the application code.

Example:
Bad:
Application Code
↓
Password ❌

Better:
Application
↓
Secrets Manager
↓
Password
Simple ga gurthupettuko:
Secrets Manager = Don't hard-code passwords

---

11. How does an application retrieve a secret?
Answer:
An application first gets permission through an IAM role or user.
It then calls Secrets Manager to retrieve the required secret value.
Secrets Manager returns the authorized secret to the application, which can use it without storing the credential in code.
For example:
Suppose an EC2 application needs an RDS password.
The EC2 instance has an IAM role that allows GetSecretValue for that secret.
The application retrieves the password from Secrets Manager and uses it to connect to RDS.
Example:
EC2 Application
↓
IAM Permission
↓
Secrets Manager
↓
Secret Value
↓
RDS
Simple ga gurthupettuko:
Application → Secrets Manager → Secret Value

---

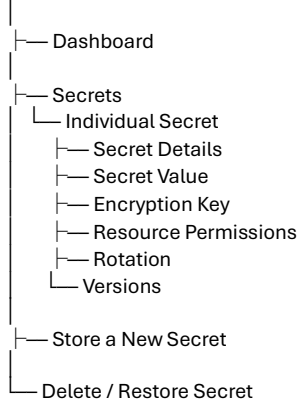
12. What happens when you delete a Secret?
Answer:
When we delete a secret, AWS Secrets Manager normally schedules it for deletion instead of immediately removing it permanently.
A recovery period can be configured so that the secret can be restored before permanent deletion.
This helps protect against accidental deletion.
For example:
Suppose someone accidentally deletes a production database secret.
Instead of losing it immediately, Secrets Manager keeps it in the scheduled deletion state for the configured recovery period.
An administrator can restore the secret during that period.
Example:
Secret
↓
Delete
↓
Scheduled for Deletion
↓
Recovery Period
↓
Permanent Deletion
Simple ga gurthupettuko:
Delete Secret = Scheduled deletion + Recovery option

---

13. What is the difference between Secrets Manager and Systems Manager Parameter Store?
Answer:
Both services can store configuration information and sensitive values, but Secrets Manager is designed specifically for secret management.
Secrets Manager provides features such as automatic secret rotation and dedicated secret lifecycle management.
Parameter Store is commonly used for configuration parameters and can also store secure strings.
For example:
Suppose we need to store an application configuration value such as an environment name.
Parameter Store can be suitable for that, while an RDS password that needs automatic rotation is a common Secrets Manager use case.
Example:
Configuration

↓
Parameter Store

Passwords / Secrets
↓
Secrets Manager
Simple ga gurthupettuko:
Parameter Store = Configuration | Secrets Manager = Secrets

Final Console Memory Map
AWS Secrets Manager


One-line memory:
Secrets Manager = Store + Encrypt + Control Access + Rotate + Retrieve Secrets
ChatGPT is AI and can make mistakes.